Logs everywhere

- **PCI DSS**
- GLBA
- Sarbanes Oxley
- **ISO 27001**
- HIPAA
- GDPR
- You name any regulations they all require log reviews and their protection.



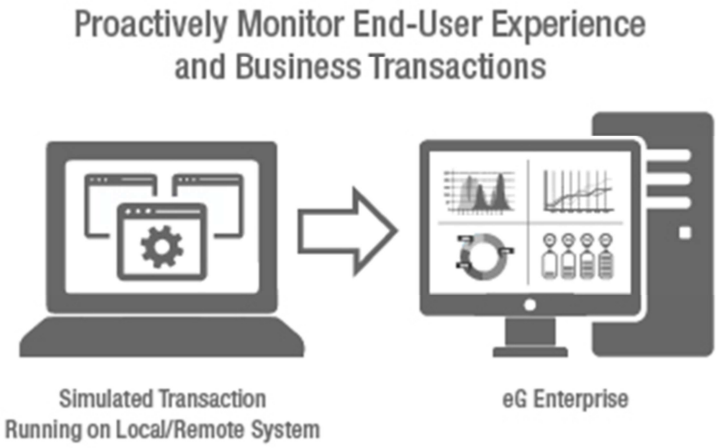
© 2018 Al-Nafi. All Rights Reserved.

Logs Security

- Print and read this document https://www.pcisecuritystandards.org/documents/Effective-Daily-Log-Monitoring-Guidance.pdf
- This will be covered in detail during ISO 27001 and PCI DSS courses.

© 2018 Al-Nafi. All Rights Reserved.

Synthetic Transactions



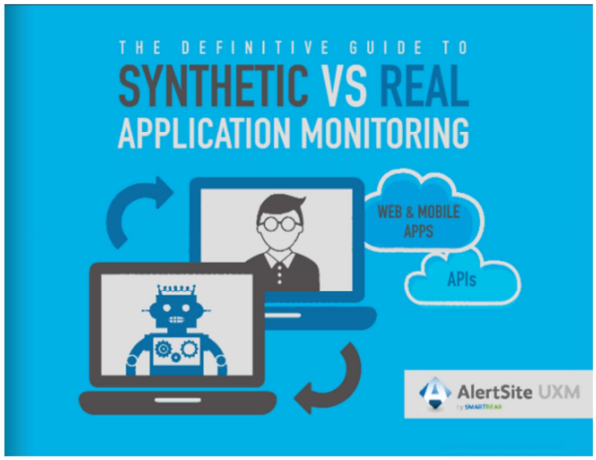
© 2018 Al-Nafi. All Rights Reserved.

"**Synthetic** monitoring (also known as active monitoring or proactive monitoring) is website monitoring that is done using a Web browser emulation or scripted recordings of Web **transactions**. Behavioral scripts (or paths) are created to simulate an action or path that a customer or end-user would take on a site."

It has two types

- Real User Monitoring (RUM)
- Synthetic Performance Monitoring

Synthetic monitoring



© 2018 Al-Nafi. All Rights Reserved.

Synthetic monitoring (also known as ***active monitoring*** *or* ***proactive monitoring***) is a monitoring technique that is done by using an emulation or scripted recordings of transactions. Behavioral scripts (or paths) are created to simulate an action or path that a customer or end-user would take on a site, application or other software (or even hardware). Those paths are then continuously monitored at specified intervals for performance, such as: functionality, availability, and response time measures.
Synthetic monitoring is valuable because it enables a webmaster or an IT/Operations professional to identify problems and determine if a website or application is slow or experiencing downtime before that problem affects actual end-users or customers. This type of monitoring does not require actual traffic, thus the name synthetic, so it enables companies to test applications 24x7, or test new applications prior to a live customer-facing launch. This is usually a good complement when used with passive monitoring to help provide visibility on application health during off peak hours when transaction volume is low.[1]
When combined with traditional APM tools, synthetic monitoring can provide

deeper visibility into end-to-end performance, regardless of where applications are running.[2]
Because synthetic monitoring is a simulation of typical user behavior or navigation through a website, it is often best used to monitor commonly trafficked paths and critical business processes. Synthetic tests must be scripted in advance, so it is not feasible to measure performance for every permutation of a navigational path an end-user might take. This is more suited for passive monitoring.
Synthetic testing is useful for measuring uptime, availability and response time of critical pages and transaction (how a site performs from all geographies) but doesn't monitor or capture actual end-user interactions, see Website monitoring. This is also known as Active monitoring that consists of synthetic probes and Web robots to help report on system availability and predefined business transactions.[3]

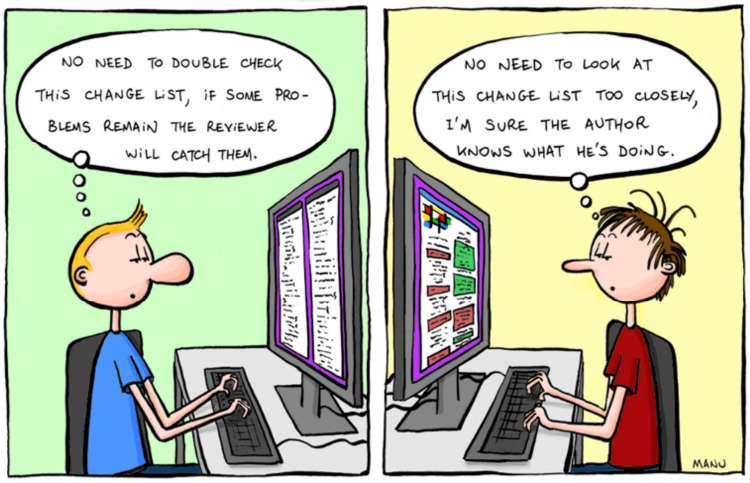
Source : https://en.wikipedia.org/wiki/Synthetic_monitoring

Types of Monitoring
Website Monitoring: Website monitoring uses synthetic transactions to perform HTTP requests to check availability and to measure performance of a web page, website, or web application.

Database Monitoring: Database monitoring using synthetic transactions monitors the availability of a database.

TCP Port Monitoring: A TCP port synthetic transaction measures the availability of your website, service, or application; you can specify the server and TCP port for
Operations Manager to monitor.

Code Review and Testing



© 2018 Al-Nafi. All Rights Reserved.

As per ISO 27002:2013 Standard

14.2 Security in development and support processes
Objective: To ensure that information security is designed and implemented within the development lifecycle of information systems.

14.2.1 Secure development policy
Control
Rules for the development of software and systems should be established and applied to developments within the organization.

Implementation guidance
Secure development is a requirement to build up a secure service, architecture, software and system. Within a secure development policy, the following aspects should be put under consideration:
a) security of the development environment;
b) guidance on the security in the software development lifecycle:

1) security in the software development methodology;
2) secure coding guidelines for each programming language used;

c) security requirements in the design phase;
d) security checkpoints within the project milestones;
e) secure repositories;
f) security in the version control;
g) required application security knowledge;
h) developers' capability of avoiding, finding and fixing vulnerabilities.

Secure programming techniques should be used both for new developments and in code re-use scenarios where the standards applied to development may not be known or were not consistent with current best practices. Secure coding standards should be considered and where relevant mandated for use. Developers should be trained in their use and testing and code review should verify their use. If development is outsourced, the organization should obtain assurance that the external party complies with these rules for secure development.

Other information
Development may also take place inside applications, such as office applications, scripting, browsers and databases.

During Planning and Design
While a security review of the architecture and threat modeling are not security testing methods, they are an important prerequisite for subsequent security testing efforts, and the security practitioner should be aware of the options available to them. The following is a consideration of the prerequisites and benefits of architecture security review and threat modeling:

Architecture security review: A manual review of the product architecture to ensure that it fulfills the necessary security requirements:
o Prerequisites: Architectural model
o Benefit: Detecting architectural violations of the security standard

Threat modeling: A structured manual analysis of an application specific business case or usage scenario. This analysis is guided by a set of precompiled security threats:

o Prerequisites: Business Case or Usage Scenario
o Benefits: Identification of threats, including their impact and potential countermeasures specific to the development of the software product

These methods help to identify the attack surface and, thus, the most critical components. This allows a focusing of the security testing activities to ensure they are as effective as possible.

What is SAST



© 2018 Al-Nafi. All Rights Reserved.

What is SAST?
Static application security testing (SAST), or static analysis, is a testing methodology that analyzes source code to find security vulnerabilities that make your organization's applications susceptible to attack. SAST scans an application before the code is compiled. It's also known as white box testing.

What problems does SAST solve?
SAST takes place very early in the software development life cycle (SDLC) as it does not require a working application and can take place without code being executed. It helps developers identify vulnerabilities in the initial stages of development and quickly resolve issues without breaking builds or passing on vulnerabilities to the final release of the application. SAST tools give developers real-time feedback as they code, helping them fix issues before they pass the code to the next phase of the SDLC. This prevents security-related issues from being considered an afterthought. SAST tools also provide graphical representations of the issues found, from source to sink. These help you navigate the code easier. Some tools point out the exact location of

vulnerabilities and highlight the risky code. Tools can also provide in-depth guidance on how to fix issues and the best place in the code to fix them, without requiring deep security domain expertise.

Developers can also create the customized reports they need with SAST tools; these reports can be exported offline and tracked using dashboards. Tracking all the security issues reported by the tool in an organized way can help developers remediate these issues promptly and release applications with minimal problems. This process contributes to the creation of a secure SDLC. It's important to note that SAST tools must be run on the application on a regular basis, such as during daily/monthly builds, every time code is checked in, or during a code release.

Why is SAST an important security activity?

Developers dramatically outnumber security staff. It can be challenging for an organization to find the resources to perform code reviews on even a fraction of its applications. A key strength of SAST tools is the ability to analyze 100% of the codebase. Additionally, they are much faster than manual secure code reviews performed by humans. These tools can scan millions of lines of code in a matter of minutes. SAST tools automatically identify critical vulnerabilities—such as buffer overflows, SQL injection, cross-site scripting, and others—with high confidence. Thus, integrating static analysis into the SDLC can yield dramatic results in the overall quality of the code developed.

What are the key steps to run SAST effectively?

There are six simple steps needed to perform SAST efficiently in organizations that have a very large number of applications built with different languages, frameworks, and platforms.

Finalize the tool. Select a static analysis tool that can perform code reviews of applications written in the programming languages you use. The tool should also be able to comprehend the underlying framework used by your software.

Create the scanning infrastructure, and deploy the tool. This step involves handling the licensing requirements, setting up access control and authorization, and procuring the resources required (e.g., servers and databases) to deploy the tool.

Customize the tool. Fine-tune the tool to suit the needs of the organization. For

example, you might configure it to reduce false positives or find additional security vulnerabilities by writing new rules or updating existing ones. Integrate the tool into the build environment, create dashboards for tracking scan results, and build custom reports.

Prioritize and onboard applications. Once the tool is ready, onboard your applications. If you have a large number of applications, prioritize the high-risk applications to scan first. Eventually, all your applications should be onboarded and scanned regularly, with application scans synced with release cycles, daily or monthly builds, or code check-ins.

Analyze scan results. This step involves triaging the results of the scan to remove false positives. Once the set of issues is finalized, they should be tracked and provided to the deployment teams for proper and timely remediation.

Provide governance and training. Proper governance ensures that your development teams are employing the scanning tools properly. The software security touchpoints should be present within the SDLC. SAST should be incorporated as part of your application development and deployment process.

What tools can be used for SAST?

Synopsys offers the most comprehensive solution for integrating security and quality into your SDLC and supply chain.

Coverity Static Application Security Testing finds critical defects and security weaknesses in code as it's written. It provides full path coverage, ensuring that every line of code and every potential execution path is tested. Through a deep understanding of the source code and the underlying frameworks, it provides highly accurate analysis, so developers don't waste time on a large volume of false positives.

Coverity scales to accommodate thousands of developers and can analyze projects with more than 100 million lines of code with ease. It can be rapidly integrated with critical tools and systems that support the development process, such as source control management, build and continuous integration, bug tracking, and application life cycle management (ALM) solutions, as well as IDEs.

SAST in IDE (Code Sight) is a real-time, developer-centric SAST tool. It scans for and identifies vulnerabilities as developers code. Code Sight integrates into the integrated development environment (IDE), where it identifies security vulnerabilities and provides guidance to remediate them.

How is SAST different from DAST?

Organizations are paying more attention to application security, owing to the rising number of breaches. They want to identify vulnerabilities in their applications and mitigate risks at an early stage. There are two different types of application security testing—SAST and dynamic application security testing (DAST). Both testing methodologies identify security flaws in applications, but they do so differently.

Static source code analysis

- Watch this video https://youtu.be/PF4X2vNv32U

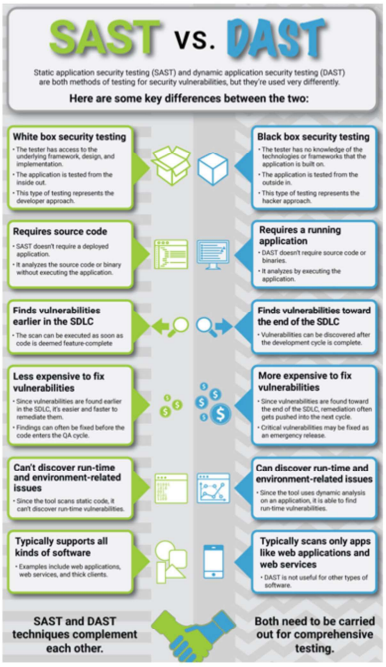
© 2018 Al-Nafi. All Rights Reserved.

Static Source Code Analysis (SAST) and manual code review:
Analysis of the application source code for finding vulnerabilities without executing the application:

- Prerequisites: Application source code
- Benefits: Detection of insecure programming, outdated libraries, and misconfigurations

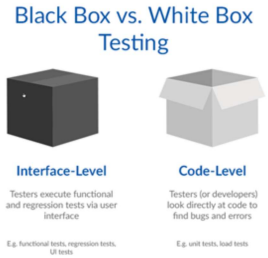
Static binary code analysis and manual binary review: Analysis of the compiled application (binary) for finding vulnerabilities without executing the application. In general, this is like the source
code analysis but is not as precise and fix recommendations typically cannot be provided.

Static Source Code Analysis (SAST) and manual code review



© 2018 Al-Nafi. All Rights Reserved.

Black-box testing vs. white-box testing



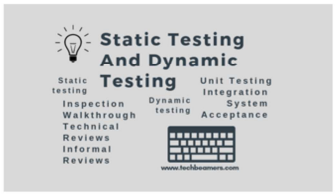
Black Box Testing	White Box Testing
1. Black box testing techniques are also called functional testing techniques.	1. White box testing techniques are also called structural testing techniques.
2Black Box Testing is a software testing method in which the internal structure/ design/ implementation of the item being tested is NOT known to the tester	2. White Box Testing is a software testing method in which the internal structure/ design/ implementation of the item being tested is known to the tester.
3. It is mainly applicable to higher levels of testing such as Acceptance Testing and System Testing	3. Mainly applicable to lower levels of testing such as Unit Testing and Integration Testing
4. Black box testing is generally done by Software Testers	4. White box testing is generally done by Software Developers
5. Programming knowledge is not required	5. Programming knowledge is required
6. Implementation knowledge is not required.	6. Implementation knowledge is required

© 2018 Al-Nafi. All Rights Reserved.

In black-box testing, the tested system is used as a black box, i.e., no internal details of the system implementation are used. In contrast, white-box testing takes the internal system details (e.g., the source code) into account.

Dynamic testing vs. static testing

Static Testing	Dynamic Testing
1. Static Testing is white box testing which is done at early stage of development life cycle. It is more cost effective than dynamic testing	1. Dynamic Testing on the other hand is done at the later stage of development lifecycle.
2. Static Testing Methods include Walkthroughs, code review.	2. Dynamic testing involves functional and nonfunctional testing
3. It is done before code deployment and without execution of code.	3. It is done after code deployment with the execution of codes.
4. This type of testing comes under Verification.	4. This type of testing comes under Validation.
5. Static testing achieves 100% statement coverage in a relatively short time.	5. Dynamic testing may involve running several test cases, each of which may take longer time.
6. Static testing is about prevention.	6. Dynamic testing is about cure.
7. In Static Testing techniques a checklist is prepared for testing process.	7. In Dynamic Testing technique the test cases are executed.



© 2018 Al-Nafi. All Rights Reserved.

Static Testing And Dynamic Testing – Understand the Difference
Software Testing Tutorial | By Meenakshi Agarwal

Static testing and dynamic testing are essential testing techniques meant for developers and testers for use during the Software Development. These are unique validation methods which the organization must decide after due analysis which one to practice for software verification. Since your objective is to get the maximum benefit from these type of testing, hence pick the right tool most suited for your needs. In this tutorial, you'll learn the pros and cons of each of these **type of testing**.

Static Testing And Dynamic Testing
Static Testing:

In Static Testing, it does not require executing the code. You can perform it manually or by using a set of tools. This testing type covers the analysis of the source, review of specification documents, also the design description

documents. In this, the testers provide review comments on each of the doc reviewed. If the application is yet not operational and hasn't implemented the user interface, then you can execute a security analysis to observe it under the runtime-less configuration.

While doing static testing, a tester or a developer can go on to look for the bugs, buffer overflows, and probably identify the vulnerable code in the system. To begin such testing, you don't need to wait for finishing the entire application development. It can get started in an early phase of the development lifecycle. And testers can begin reviewing the code, scripts, requirements, test cases, or any related doc whichever is available at the point of time.

Static Testing Techniques:

1. Inspection:

The principal moto of this Type of Testing is to identify defects in the early stage of the software development cycle. The team can start with the inspection of any of the artifacts as mentioned earlier such the code or the test cases or the product docs walkthroughs. It requires a moderator for organizing the review sessions. Since the inspection is a formal type of review, so the moderator needs to prepare a checklist of what to go through and what not.

2. Walkthrough:

Another technique is the walkthrough. It requires the owner of the document to explain the work done. The attendees can place their queries and another person allocated as the scribe has to
record the points in the notes.

3. Technical Reviews:

In this Static Testing method, the team carries out the Technical Scrutiny of the code written to check whether it meets the coding guidelines and standards. In general, the testing artifacts such as a testing plan, validation strategy, and the automation scripts get also reviewed in this session.

4. Informal Reviews:

Static Testing technique in which the docs get scrutinized informally. Also, the participants provide informal comments during the meeting.

Dynamic Testing:
Dynamic testing gets performed after the application has reached into the operational mode. It gets executed in the real runtime environment set up by the QA. While the code behind the app is running, the tester supplies the required input and waits for the result. After that, he/she matches the output with the expected outcome.
That's how the testers inspect the functional behavior of the application, track the system RAM, CPU usage, its response time, and the performance of the overall software. Dynamic testing has another name as validation testing. It can either be Functional Testing as well as non-functional testing.

Types Of Dynamic Testing Techniques:

1. Unit Testing:
This testing happens mostly at the developer's end. The essential artifacts which get tested are the source code of the application's various modules.

2. Integration Testing:
The purpose of this technique is to verify the interfacing between two or more modules once they get tied up together.

3. System Testing:
This testing gets done on the entire software with all modules working.

4. Acceptance Testing:
This testing runs the validation keeping the user's point of view in mind.

In software development lifecycle, both the Static Testing and Dynamic Testing are essential to certify the application functionality. Each of these has its strength and weakness which you should be aware.

Static Testing Vs. Dynamic Testing:
1. Static Testing belongs to the white box testing. It gets performed at an early stage of development. It incurs a lower cost as compared to the dynamic testing. Vs Dynamic Testing gets performed at the final stage of the development process.
2. Static testing has better line coverage than the dynamic testing in the short duration. Vs Dynamic Testing has lower no. Of line coverage as it examines only

a smaller part of the code.
3. Static testing occurs before the application is ready for the deployment. Vs. Dynamic Testing happens after the code got deployed.
4. Static testing gets done in the verification stage. Vs. Dynamic testing completes in the validation stage.
5. No execution happens in the static testing. Vs. Dynamic testing requires the code execution.
6. Static testing produces the analysis of the code along with the documentation. Vs. Dynamic Testing reports the bottlenecks in the application.
7. In Static Testing, the team prepares a checklist describing the testing process. Vs. In Dynamic Testing, the test cases get executed.
8. Static Testing Methods are Walkthroughs and code reviews. Vs. Dynamic testing majorly has the functional and nonfunctional validation.

Summary – Static Testing And Dynamic Testing
In any Software development methodology, both the Verification and Validation process get carried out to certify that the final software has all the requirements implemented correctly.
Static testing scrutinizes the application code without any execution. It lies under the umbrella of Verification. The testers have got multiple Static testing techniques such as Inspection, Walkthrough, Technical and Informal reviews, etc.

On the contrary, Dynamic testing validates the working product. It lies under the umbrella of Validation. The standard Dynamic testing techniques are Unit Testing, Integration Testing, System or Stabilization Testing and User Acceptance Testing. Here, the product gets validated by both functional and non-functional aspects.

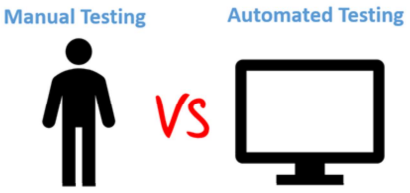
Manual testing vs. automated testing

Manual Testing

- Manual testing is not accurate at all times due to human error, hence it is less reliable.
- Manual testing is time-consuming, taking up human resources.
- Investment is required for human resources.
- Manual testing is only practical when the test cases are run once or twice, and frequent repetition is not required.
- Manual testing allows for human observation, which may be more useful if the goal is user-friendliness or improved customer experience.

Automated Testing

- Automated testing is more reliable, as it is performed by tools and/or scripts.
- Automated testing is executed by software tools, so it is significantly faster than a manual approach.
- Investment is required for testing tools.
- Automated testing is a practical option when the test cases are run repeatedly over a long time period.
- Automated testing does not entail human observation and cannot guarantee user-friendliness or positive customer experience.



© 2018 Al-Nafi. All Rights Reserved.

Manual vs. Automated pentesting



MANUAL PENETRATION TESTING

- In-depth penetration testing
- Performed by security experts often working as external consultants
- Not integrated into the development process

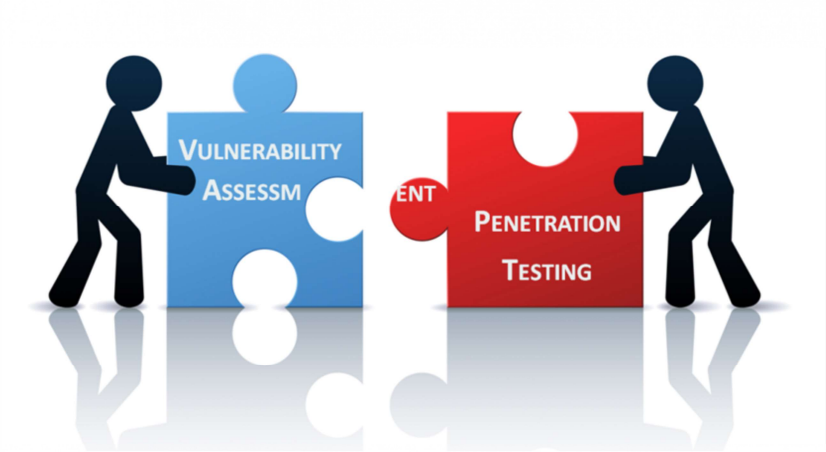
AUTOMATED SECURITY TESTING

- Regular security testing using a fully automated and up-to-date service
- Increases knowledge across the organisation
- Easy-to-understand reports
- Complements compliance and support teams security efforts.



© 2018 Al-Nafi. All Rights Reserved.

Automated vulnerability scanners



© 2018 Al-Nafi. All Rights Reserved.

Automated vulnerability scanners: Test an application for the use of system components or configurations that are known to be insecure. For this, predefined attack patterns are executed as well as system fingerprints are analyzed:

- o Benefits: Detection of well-known vulnerabilities, i.e., detection of outdated frameworks and misconfigurations

Fuzz testing tools



© 2018 Al-Nafi. All Rights Reserved.

Source Wikipedia:

Fuzzing or **fuzz testing** is an automated software testing technique that involves providing invalid, unexpected, or random data as inputs to a computer program. The program is then monitored for exceptions such as crashes, failing built-in code assertions, or potential memory leaks. Typically, fuzzers are used to test programs that take structured inputs. This structure is specified, e.g., in a file format or protocol and distinguishes valid from invalid input. An effective fuzzer generates semi-valid inputs that are "valid enough" in that they are not directly rejected by the parser, but do create unexpected behaviors deeper in the program and are "invalid enough" to expose corner cases that have not been properly dealt with.

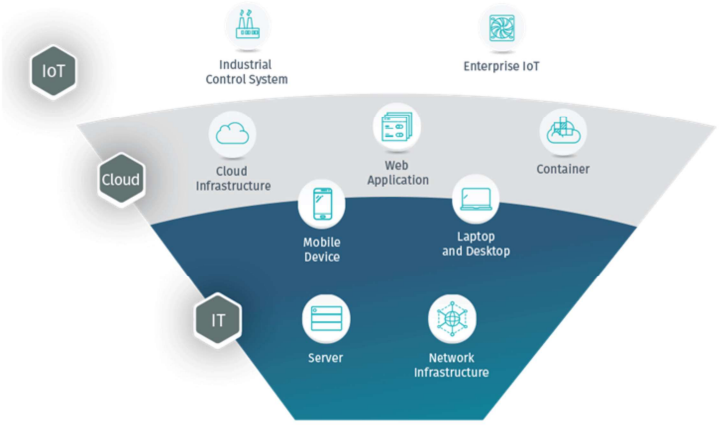
For the purpose of security, input that crosses a trust boundary is often the most interesting.[1] For example, it is more important to fuzz code that handles the upload of a file by any user than it is to fuzz the code that parses a configuration file that is accessible only to a privileged user.

Security testing methods or tools.

- Attack surface
- Application type security testing methods
- Quality of results and usability
- Supported technologies
- Performance and resource utilization

© 2018 Al-Nafi. All Rights Reserved.

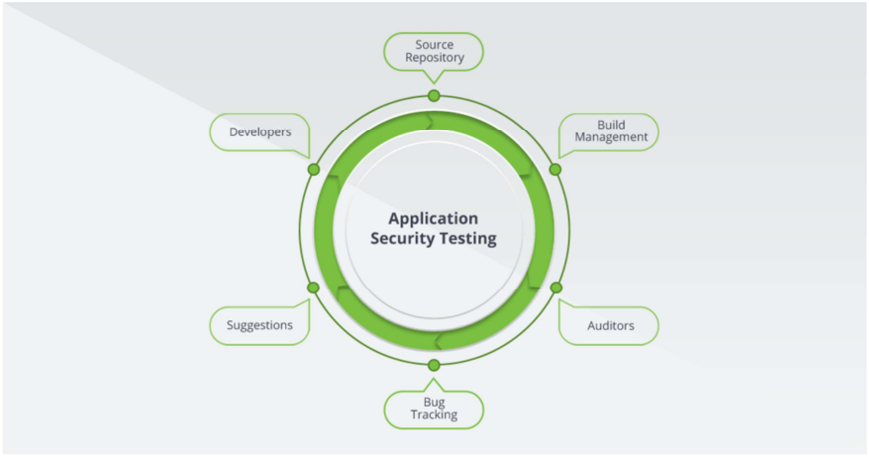
Attack surface



© 2018 Al-Nafi. All Rights Reserved.

The **attack surface** of a software or hardware environment is the sum of the different points (the "**attack** vectors") where an unauthorized user (the "attacker") can try to enter data to or extract data from an environment. Keeping the **attack surface** as small as possible is a basic security measure.

Application type security testing methods



© 2018 Al-Nafi. All Rights Reserved.

Different security testing methods behave differently when applied to different application types

Quality of results and usability



© 2018 Al-Nafi. All Rights Reserved.

Security testing techniques and tools differ in usability (e.g., fix recommendations) and quality (e.g., false positives rate)

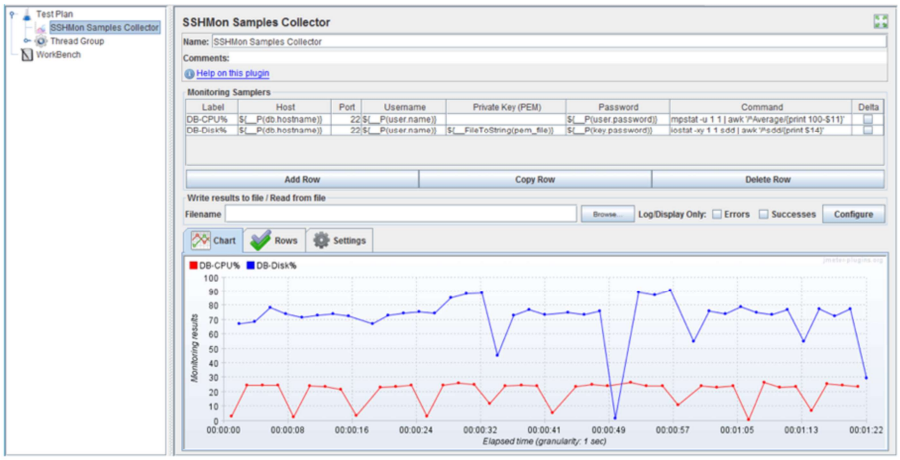
Supported technologies



© 2018 Al-Nafi. All Rights Reserved.

Security testing tools usually only support a limited number of technologies (e.g., programming languages), and if a tool supports multiple technologies, it does not necessarily support all of them equally well

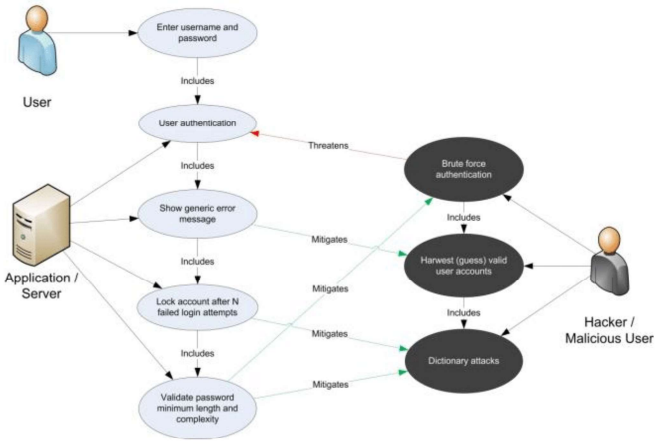
Performance and resource utilization



© 2018 Al-Nafi. All Rights Reserved.

Different tools and methods require different computing power or different manual efforts

Misuse Case Testing



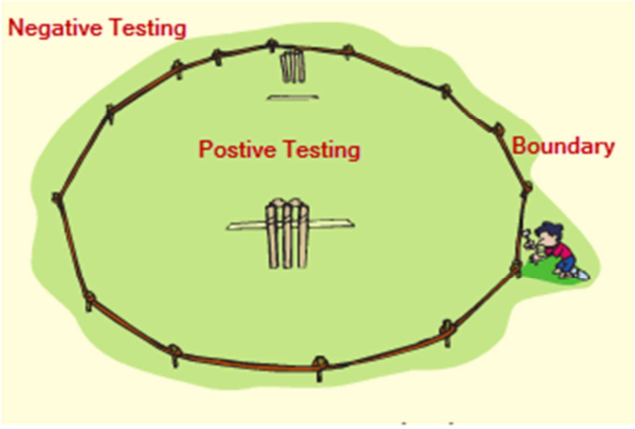
© 2018 Al-Nafi. All Rights Reserved.

Misuse case is a business process modeling tool used in the software development industry. The term *Misuse Case* or *mis-use case* is derived from and is the inverse of use case.[1] The term was first used in the 1990s by Guttorm Sindre of the Norwegian University of Science and Technology, and Andreas L. Opdahl of the University of Bergen, Norway. It describes the process of executing a malicious act against a system, while use case can be used to describe any action taken by the system.[2]

Some misuse cases occur in highly specific situations, whereas others continually threaten systems. For instance, a car is most likely to be stolen when parked and unattended; whereas a web server might suffer a denial-of-service attack at any time. You can develop misuse and use cases recursively, going from system to subsystem levels or lower as necessary. Lower-level cases can highlight aspects not considered at higher levels, possibly forcing another analysis. The approach offers rich possibilities for exploring, understanding, and validating the requirements in any direction. Drawing the agents and misuse cases explicitly helps to focus the attention of the security

practitioner on the elements of the scenario.

Explore Negative Testing



© 2018 Al-Nafi. All Rights Reserved.

In contrast to a positive test (that determines that a system works as expected, and with any error fails the test); a negative test is designed to provide evidence of the application behavior if there is unexpected or invalid data. Any provocation of application failure is designed to surface in the test rather than once the application is approved for production. An optimal response for an application to a negative test is to gracefully reject the unexpected or invalid data without crashing. While exceptions and error conditions are expected in negative tests
they are not expected in positive tests. It is optimal to combine a range of positive and negative test to run on an application for thorough examination of behavior.